Loops handling by BPF verifier and what should we about it?

Overview

This talk is a bit of a hodgepodge of loop related things I worked on this year in close cooperation with Alexei:

- Ongoing work on scalar evolution.
- ~~“bpf: simple DFA-based live registers analysis” (Mar 2025, link).~~
- “bpf: propagate read/precision marks over state graph backedges” (Jun 2025, link).
- “bpf: replace path-sensitive with path-insensitive live stack analysis” (Sep 2025, link).

Scalar Evolution

Verify bounded loops avoiding verifier states explosion.

Scalar Evolution: basic idea

For all variables changed inside a loop, the analysis attempts to recover an algebraic expression parameterized by the initial values of the variables and the iteration number.

```
while (i < 10) {
  arr[i] = ...;
  i += 2;
}
```

$i = i_0 + 2 * k$

i_0 the initial value of `i`

k_0 the loop iteration number

Scalar Evolution: basic idea

Use these algebraic expressions and information about what conditions guard loop backedges, to estimate the number of loop iterations.

```
while (i < 10) {
  arr[i] = ...;
  i += 2;
}
```

Solve the inequality $i_0 + 2*k < 10$ with respect to k.

Scalar Evolution: collecting algebraic expressions

Algebraic expressions collection is a three step process[1]:

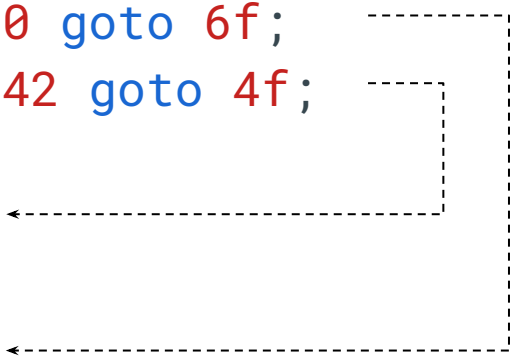
1. Do a single pass over the loop body, collecting expressions describing variable values relative to the values at the loop header.
   Accumulate these expressions at the loop header.
2. Infer recursive relations from the values accumulated at the loop header.
3. Using expressions from step (1) and recursive relations from step (2) infer recursive expressions for variables at each statement within the loop body.

---

1. R. van Engelen, “The CR# Algebra and its Application in Loop Analysis and Optimization” (link)

Collecting algebraic expressions: example

```
   ...
1: if r0 > 10 goto 6f;
2: if r2 != 42 goto 4f;
3: r1 += 1;
4: r0 += 1;
5: goto 1b;
6: ...
```

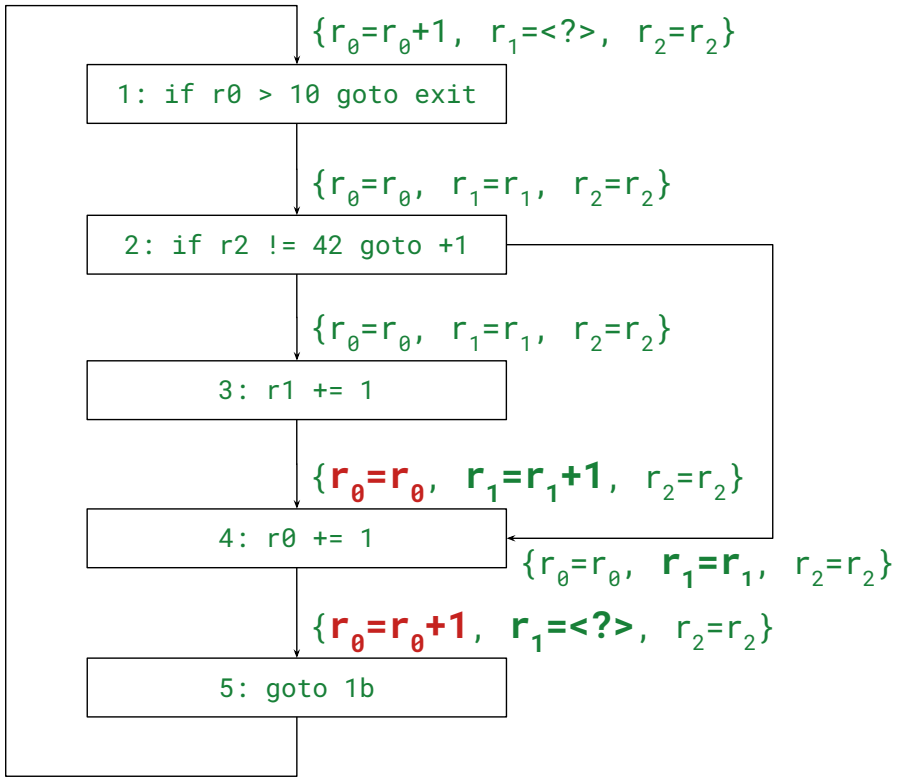


`r0` is always updated within the loop body,
`r1` is updated conditionally.

Collecting algebraic expressions: example

Step 1: collect algebraic expressions.

```
    ...
1:  if r0 > 10 goto 6f;
2:  if r2 != 42 goto 4f;
3:  r1 += 1;
4:  r0 += 1;
5:  goto 1b;
6:  ...
```



Collecting algebraic expressions: example

Step 2: infer recursive relations.

```
   ...
1: if r0 > 10 goto 6f;
2: if r2 != 42 goto 4f;
3: r1 += 1;
4: r0 += 1;
5: goto 1b;
6: ...
```

Algebraic expression at loop header:

$\{r_0=r_0+1, r_1=\langle ?\rangle, r_2=r_2\}$

Corresponds to recursive relation:

$\{r_0=r_0^{initial}+k,\ r_1=\langle ?\rangle, r_2=r_2^{initial}\}$

Collecting algebraic expressions: example

Step 3: propagate recursive relations.

```
   ...
1: if r0 > 10 goto 6f;
2: if r2 != 42 goto 4f;
3: r1 += 1;
4: r0 += 1;
5: goto 1b;
6: ...
```

Recursive relation at loop header: $r_0 = r_0^{initial} + k$

Algebraic expression at (5): $r_0 = r_0 + 1$

Recursive relation at (5): $r_0 = r_0^{initial} + 1 + k$

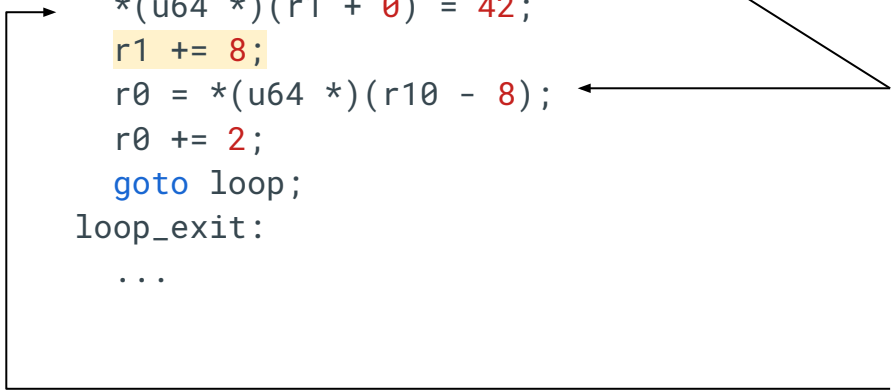
Scalar Evolution: disassembly analysis

Unlike C compilers, the verifier operates at the assembly level, so it needs to track values through stack spills and fills.
This poses a recurring problem:

- Tracking stack spills and fills requires range information.
- Range information requires stack spills and fills tracking.

Scalar Evolution: disassembly analysis

```
  ...
loop:
  if r0 > 10 goto loop_exit;
  *(u64 *)(r10 - 8) = r0;
  r0 = r10;
  r0 += r1;
  *(u64 *)(r1 + 0) = 42;
  r1 += 8;
  r0 = *(u64 *)(r10 - 8);
  r0 += 2;
  goto loop;
loop_exit:
  ...
```



Algebraic expression for `r0` is needed to estimate the number of loop iterations.

Correspondence between `r0` and stack slot `r10-8` has to hold for `r0` algebraic expression to remain valid.

Might overwrite spilled value for `r0`, depending on the initial value of `r1` and maximal number of loop iterations.

Scalar Evolution: Value Set Analysis

The same problem is faced by decompilers under the name “Variable Detection”[1]. It appears that the state of the art approach to this problem is named Value Set Analysis (VSA)[2]:

- Uses interval abstract domain extended with value stride: `{region, lower_bound, upper_bound, step}`.
- Combines pointers and scalars reasoning.
- Classic abstract interpretation analysis with widening and narrowing steps.

This is both very similar and significantly different compared to the current verifier logic.

---

1. Decompilation Wiki: Variable Detection (link)
2. G. Balakrishnan, “WYSINWYX: What You See Is Not What You eXecute” (link)

Workaround: separate spill base register

Sidestep the spill tracking problem completely by using a separate base register for stack spills:

- Extend instruction set with register r_{11}.
- Restricts usage of r_{11} to the two instructions:
  - `*(u64 *)(r11 - 8*k) = rX`
  - `rX = *(u64 *)(r11 - 8*k)`
- After verification is finished convert access to r_{11} as access to r_{10} by adjusting all offsets by subprogram stack depth.
- Modify LLVM BPF backend to use r_{11} as a base register for stack spills instead of r_{10}.

Workaround: separate spill base register

Original	r_{11} spill base	After lowering r_{11} to r_{10}

`  r1 = -56;`
`loop:`
`  if r0 > 10 goto lexit;`
`  *(u64 *)(r10 - 8) = r0;`
`  r0 = r10;`
`  r0 += r1;`
`  *(u64 *)(r1 + 0) = 42;`
`  r1 += 8;`
`  r0 = *(u64 *)(r10 - 8);`
`  r0 += 2;`
`  goto loop;`
`lexit:`
`  ...`
	`  r1 = -56;`
	`loop:`
	`  if r0 > 10 goto lexit;`
	`  *(u64 *)(r11 - 8) = r0;`
	`  r0 = r10;`
	`  r0 += r1;`
	`  *(u64 *)(r1 + 0) = 42;`
	`  r1 += 8;`
	`  r0 = *(u64 *)(r11 - 8);`
	`  r0 += 2;`
	`  goto loop;`
	`lexit:`
	`  ...`
		`  r1 = -56;`
		`loop:`
		`  if r0 > 10 goto lexit;`
		`  *(u64 *)(r11 - 64) = r0;`
		`  r0 = r10;`
		`  r0 += r1;`
		`  *(u64 *)(r1 + 0) = 42;`
		`  r1 += 8;`
		`  r0 = *(u64 *)(r11 - 64);`
		`  r0 += 2;`
		`  goto loop;`
		`lexit:`
		`  ...`

While this approach is simple to implement, the obvious downside is the lack of backwards compatibility. Prototypes: LLVM, Kernel.

Another workaround: verifier states with assumptions

- Run scalar evolution analysis before the main verification pass.
- Build algebraic expressions assuming that each direct stack access (store `*(u64*)(r10-8*k)=rX`, load `rX=*(u64*)(r10-8*k)`), is a spill or fill to an unaliased location.
- Attached the stack slots set to each algebraic expression.
- Use algebraic expressions to compute loop iteration bounds on the main verification pass, record under what assumptions these bounds hold.
- If any of the stack slots assumed to be unaliased are modified indirectly within the loop body, the assumptions become invalid, thus abandon the corresponding verification path.

Verifier states with assumptions: example

```
    ...
loop:
  if r0 > 10 goto loop_exit; // a
  *(u64 *)(r10 - 8) = r0;
  r0 = r10;
  r0 += r1;
  *(u64 *)(r1 + 0) = 42;      // b
  r1 += 8;
  r0 = *(u64 *)(r10 - 8);
  r0 += 2;
  goto loop;
loop_exit:
  ...
```

at (a) $r_0 = r_0^{initial} + 2*k$,
assuming direct access to slot $r_{10} - 8$

at (b) $r_1 = r_1^{initial} + 8*k$

Verifier states with assumptions: example

```
  ... r0=0, r1=-56 ...
loop:
  if r0 > 10 goto loop_exit; // a
  *(u64 *)(r10 - 8) = r0;
  r0 = r10;
  r0 += r1;
  *(u64 *)(r1 + 0) = 42;      // b
  r1 += 8;                    // c
  r0 = *(u64 *)(r10 - 8);
  r0 += 2;
  goto loop;                  // d
loop_exit:
  ...
```

At point (a) infer loop iteration bounds: $0 \le k \le 5$.
Proceeds with state:
$\{r_0 = 2*k,\ r_1 = -56+8*k,\ 0 \le k \le 5,\ da: r_{10}-8\}$.

At point (b) the assumption holds: $-56 \le r_1 \le -16$,
given $r_1 = r_1^{initial} + 8*k$.

At point (c) the verifier keeps the $r_1 = -56+8*k$:
$\{r_0 = 2*k,\ r_1 = -56+8*k, fp_{-56}..fp_{-16} = 42,\ 0 \le k \le 5,\ da: r_{10}-8\}$.

Jumping from (d) to (a) current state is not the same as at the first step, hence the verifier does one more iteration.

This time the state after jumping from (d) to (a) is the same as at step (8) and loop verification is complete.

Scalar Evolution: nested loops

While more sophisticated schemes are possible, the current prototype does the following:

- Inner loops are processed before outer loops.
- When outer loop is processed, inner loop is considered a single CFG node.
- Any registers changed within an inner loop are considered unknown in the outer loop.

Scalar Evolution: nested loops

Such scheme handles simple nested loops without problems:

```
for (i = 0; i < 10; i++) {
  for (j = 0; j < 10; j++)
    foo(i, j);
  // recursive relation for i can be tracked
}
```

But falls short for more sophisticated examples:

```
for (i = 0; i < 10; i++) {
  for (j = 0; j < 10; j++) {
    ...
    if (...) i++;
  }
  // recursive relation for i can't be tracked
}
```

Current status: work in progress

- Loop structure reconstruction. [✓]
- Backedge latches detection. [✓]
- Algebraic expressions for loop variables. [✓]
- Nested loops handling. [✓]
- SCEVs construction from algebraic expressions. [in progress]
- Stack slots tracking for algebraic expressions.
- Support for assumptions mechanics in the main verification loop.
- Use SCEVs to infer loop iteration bounds on the main verification loop.
- Support for “linear equation” domain for registers representation.

States graph accumulation

One more iterator correctness bug

A bug in read and precision marks propagation logic for states within iterator based loops triggered a significant redesign of:

- Internal iterator-based loops handling logic.
- Stack slot read marks propagation in the internal states graph.

One more iterator correctness bug: the culprit

```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

This program is unsafe, because an unaligned stack access is possible.

Before commit `c9e31900b54c` (“bpf: propagate read/precision marks over state graph backedges”) verifier accepted this program[1].

One more iterator correctness bug: the culprit

```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

This happened, because verifier cached at (6) a fully explored state $\{r_6=-32\}$ without precision mark,
before exploring the branch 9→10.

Consequently, when exploring the branch 4→5 the state $\{r_6=-31\}$ was considered equivalent.

One more iterator correctness bug: root cause

When iterator based loops are processed verifier states effectively form loops. Whenever new marks are discovered for one state in a loop, these marks need to be propagated to all other states.

Before commit `c9e31900b54c`
(“bpf: propagate read/precision marks over state graph backedges”)
this graph was not fully recorded and propagation happened only partially.

States graph accumulation: SCCs

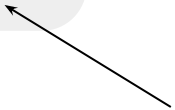
```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

Pre-compute strongly connected components (SCCs) on the program control flow graph.

States graph accumulation: entry states

```
1: r8 = bpf_get_prandom_u32();
2: r6 = -32;
3: bpf_iter_num_new(&fp[-8], 0, 10);
4: if (unlikely(bpf_get_prandom_u32()))
5:   r6 = -31;
6: for (;;) {
7:   if (!bpf_iter_num_next(&fp[-8]))
8:     break;
9:   if (unlikely(bpf_get_prandom_u32()))
10:    *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

On each verification path, the first state that enters a strongly connected component is designated as an SCC entry state.



Two entry states here,
first $\{r_6=-32\}$ then $\{r_6=-31\}$.

States graph accumulation: iterator backedges

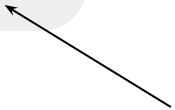
```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

State graph backedges are accumulated for a particular SCC visit.

States graph accumulation: iterator convergence

```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

As long as an SCC entry state has unexplored children, read marks are assumed to be incomplete for any state within this SCC.



Any branch within loop body 7-11 will cause entry state at (6) to have unexplored branches.

States graph accumulation: SCCs

```
 1: r8 = bpf_get_prandom_u32();
 2: r6 = -32;
 3: bpf_iter_num_new(&fp[-8], 0, 10);
 4: if (unlikely(bpf_get_prandom_u32()))
 5:   r6 = -31;
 6: for (;;) {
 7:   if (!bpf_iter_num_next(&fp[-8]))
 8:     break;
 9:   if (unlikely(bpf_get_prandom_u32()))
10:     *(u64 *)(fp + r6) = 7;
11: }
12: bpf_iter_num_destroy(&fp[-8]);
13: return 0;
```

Read marks are propagated over states graph once there are no more unexplored children for an SCC entry state.

Hence state $\{r_6=-32P\}$ will get it's precision mark from instruction (10) before the branch 4→5 and the state $\{r_6=-31\}$ are explored.

Path-insensitive live stack analysis

Path-insensitive live stack analysis

Motivation:

- Reasoning about read and precision marks propagation over verifier states graph is quite tricky.
- And is not reusable if we eventually get an oracle, predicting the number of loop iterations.

Alternative:

- Propagate read and precision marks over the program CFG.

Bonus:

- Code reuse opportunity between liveness and precision tracking.

Path-insensitive live stack analysis: status

Done:

- Read marks propagation over the program CFG

Pending:

- Precision marks propagation over the program CFG.
- Code reuse opportunity between liveness and precision tracking.

Path-insensitive live stack analysis: basic idea

Is the same as with register liveness tracking:

- For each instruction track a mask of stack slots read and written by it.
- Propagate this information via CFG using textbook data flow equations:

$$I_{out} = \bigcup_{S \in I_{succ}} \left(S_{in}\right)$$

The set of stack slots that are live after an instruction is defined as the union of the live-slot sets of all its successors.

$$I_{in} = \left(I_{out} / I_{def}\right) \cup I_{use}$$

The set of stack slots that are live before an instruction is defined as the union of its use set and the set of slots live after the instruction, excluding those defined by the instruction.

Path-insensitive live stack analysis: range information

Live stack slots cannot be computed outside of main the verification pass:

```
1: r1 = r10;
2: r1 += r2;
3: *(u64 *)(r1 + 0) = 0;
```

Identifying a set of stack slots written at (3) requires type information for r_1 and range information for r_2.

Path-insensitive live stack analysis: integration

The solution is to split the analysis is split into two phases:

- Read and write marks accumulation.
- Read and write marks propagation.

During the main verification pass:

- At each instruction accumulate the marks.
- When liveness information is needed, propagate the marks before querying.

Path-insensitive live stack analysis: subprograms

A subprogram can write to different outer frame locations depending on a callsite:

```
 1: void foo(long *p) {
 2:   ...
 3:   *p = 42;
 4:   ...
 5: }
 6: void bar(void) {
 7:   long a, b;
 8:   ...
 9:   foo(&a);
10:   ...
11:   foo(&b);
12:   ...
13: }
```

at call chain `(9, 3)` foo writes to `a`.
at call chain `(11, 3)` foo writes to `b`.

Experiments show that tracking this difference instead of blending effects at (9) and at (11) is critical for the verifier performance.

Path-insensitive live stack analysis: instances

To accommodate this the analysis tracks separate instances of `foo` for each call chain, and effects are propagated to the callsites:

```
 1: void foo(long *p) {
 2:   ...
 3:   *p = 42;
 4:   ...
 5: }
 6: void bar(void) {
 7:   long a, b;
 8:   ...
 9:   foo(&a); // must write &a
10:   ...
11:   foo(&b); // must write &b
12:   ...
13: }
```

